

Lecture 7 -- Local Search

- 我们可以把算法按成对组织：第一类按层进行，第二类按子树进行。
 - (1) 按结点深度迭代：
 - 广度优先搜索（BFS）按结点深度递增的层次进行搜索。
 - 迭代加深搜索（IDS）按结点深度递增的子树进行搜索。
 - (2) 按路径代价 + 启发函数迭代：
 - A* 搜索按路径代价与启发函数之和递增的层次进行搜索。
 - 迭代加深 A*（IDA*）按路径代价与启发函数之和递增的子树进行搜索。
 - 使用哪个代价函数？
 - 一致代价搜索（UCS）按路径代价递增的层次进行搜索。
 - 贪心最佳优先搜索按启发函数值递增的层次进行搜索。
 - A* 搜索按路径代价 + 启发函数之和递增的层次进行搜索。
- 局部搜索 Local Search
 - 背景：
 - 到目前为止看到的搜索算法都是为系统地探索搜索空间而设计的。
 - 问题通常是可观测的、确定性的、且环境已知。解通常是一系列动作。
 - 当找到目标时，从起点到该目标的路径构成问题的一个解。但根据应用不同，路径本身可能重要，也可能不重要。
 - 如果路径不重要或无法进行系统搜索，则应考虑另一类算法。
 - 在上述情况中，我们可以使用迭代改进算法，即局部搜索。
 - 在纯优化问题中也很有用，此类问题的目标是根据某个优化函数找到最优状态。
 - 示例：
 - 集成电路设计、通信网络优化等。
 - 八皇后问题：重要的是拼图的最终配置，而不是达到该配置的中间步骤。
 - 思路：保持一个“当前”状态，并尝试改进它。
 - 只移动到该节点的邻居。
 - 优点：
 - 无需维护搜索树。
 - 占用极少内存。
 - 在连续或大规模的状态空间中，常能找到足够好的解。
 - 局部搜索算法：
 - **爬山法（最陡上升/下降） Hill Climbing**
 - 贪心局部搜索

- 只查看当前节点的直接较优邻居，不再往更远处看。
- 搜索向“上坡”移动：沿着目标值增大的方向移动以找到山顶，到达峰值时终止。可能在局部最大值处终止、也可能找到全局最大值，或者陷入停滞无法继续。
- 节点既表示一个状态，也有一个对应的数值。

Hill climbing: Pseudo-code

```

function HILL-CLIMBING(initialState)
    returns State that is a local maximum

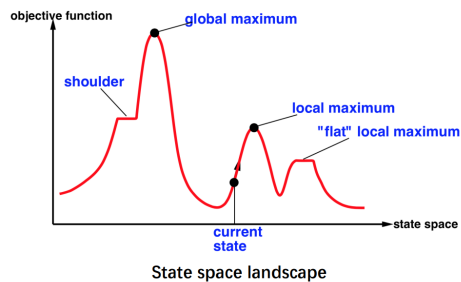
    initialize current with initialState

    loop do
        neighbor = a highest-valued successor of current

        if neighbor.value ≤ current.value:
            return current.state

        current = neighbor
  
```

- 其他爬山法变体包括：
 - 侧向移动：从平台（plateau）中逃逸，当最佳后继的值与当前状态相同时使用。
 - 随机重启爬山：克服局部极值——不断重试！（要么找到目标，要么得到若干可能解并选择最大者）。
 - 随机性爬山：在所有上坡移动中随机选择。
- 爬山法总体上有效，但依赖于状态空间地形。经过合理次数的重启，在许多实际问题中能成功。



模拟退火 Simulated Annealing:

Simulated Annealing

```

function SIMULATED-ANNEALING(problem, schedule) returns a solution state
    current ← problem.INITIAL
    for  $t = 1$  to  $\infty$  do
         $T \leftarrow \text{schedule}(t)$  温度随时间变化的函数(冷却表)
        if  $T = 0$  then return current
        next ← a randomly selected successor of current
         $\Delta E \leftarrow \text{VALUE}(\text{current}) - \text{VALUE}(\text{next})$ 
        if  $\Delta E < 0$  then current ← next //for maximization
        else current ← next only with probability  $e^{-\Delta E/T}$ 
        接受概率
  
```

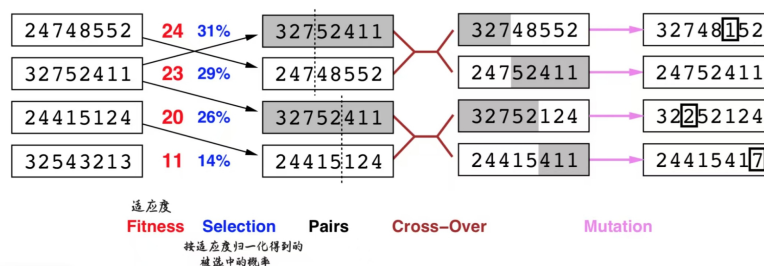
• 局部束搜索 Local beam Search

- 维护 k 个状态而不是一个。选择 k 个最佳后继，状态间传递有用信息。

• 遗传算法 Genetic Algorithms:

- GA 是随机束搜索的一个变体。
 - 随机束搜索 stochastic beam search: 随机选择 k 个后继，帮助缓解所有状态聚集在状态空间同一部分的问题。
- 个体通常用由 0 和 1（或其他有限符号）组成的字符串表示。
 - 在 8 queen 中，一个个体可以用一个由 1 到 8 的数字串表示，表示 8 列中每列皇后所在的行。
- 目标函数称为适应度函数 fitness function: 适应度值高的状态更优。
 - 在 8 queen 中，一种可能的适应度函数是“互不冲突的皇后对数”。
 - 一个解的适应度为 28。
- 从 k 个随机生成的状态开始，称为种群。每个状态称为个体。
- 个体成对随机选中用于繁殖（按一定概率选择）。
- 在染色体字符串中随机选择一个交叉点（crossover point）。
- 在交叉点处将父代拼接，生成后代（offspring）。
 - 后继状态 successor state 通过将两个父代组合生成，而不是仅修改单个状态。
- 字符串中的每个位置还会以很小的概率发生变异（mutation）。

Generate successors from pairs of states.



Genetic algorithms: Pseudo-code

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

initial population FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do** 按适应度概率随机选择

$x \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$y \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$child \leftarrow$ REPRODUCE(x, y)

if (small random probability) **then** $child \leftarrow$ MUTATE($child$)

add $child$ **to** *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN

function REPRODUCE(x, y) **returns** an individual

inputs: x, y , parent individuals 在字符串中随机选一个交叉点 c

$n \leftarrow$ LENGTH(x); $c \leftarrow$ random number from 1 to n 把 x 的前段和 y 的后段拼成 $child$

return APPEND(SUBSTRING($x, 1, c$), SUBSTRING($y, c + 1, n$))

- 关于元启发式算法 Metaheuristics 的进一步研究

- 启发式 (Heuristic)

- 针对具体问题
 - 可以被元启发式算法使用

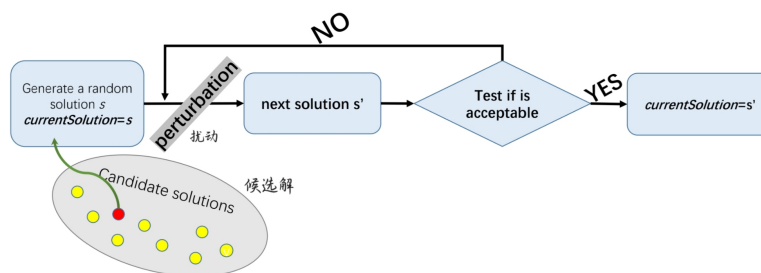
- 元启发式 (Metaheuristic)

- 与具体问题无关 (通用)
 - 可使用不同的启发式或将多种启发式组合使用

- 进化计算 (Evolutionary Computation)

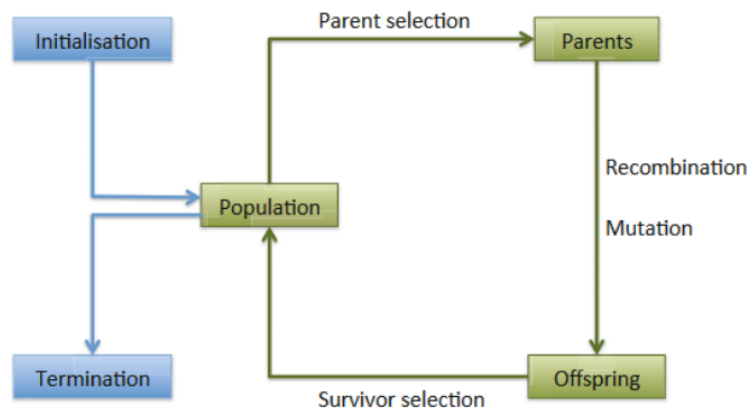
- 进化计算是研究那些借鉴自然进化思想并用于计算的系统的学科。
 - 借用的原则之一是“适者生存”。
 - 进化计算 (EC) 技术可用于优化、学习与设计任务。
 - EC 技术不需要大量领域知识即可使用；不过领域知识也可以被整合到 EC 方法中以提升效果。

- 生成-与-测试 Generate-and-Test:

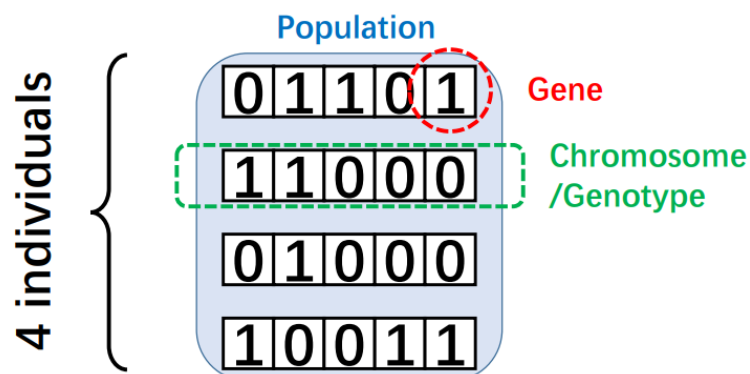


31

- 随机生成初始解，并将其记为 current solution (当前解)。
 - 通过扰动 (perturbation) 从当前解生成下一个解 (next solution)。
 - 测试新生成的解 (next solution) 是否可接受：
 - 若可接受，则将其作为当前解 (接受为 current solution)；
 - 否则保持当前解不变。
 - 如果当前解不满意，则转到步骤2；否则停止。
 - EA：基于种群的生成-与-测试
 - 生成 (Generate)：对种群内的个体进行变异和/或重组 (recombine)。
 - 测试 (Test)：从父代与子代中选择下一代。
 -



- 随机生成初始种群 $P(0)$ 。
- $i \leftarrow 0$ // 世代计数 (generation counter)
- 当未达到终止条件时循环：
- 评估 $P(i)$ 中每个个体的适应度 (Evaluate)
- 根据 $P(i)$ 中个体的适应度从 $P(i)$ 中选择父代 (Select)
- 使用交叉 (crossover) 和变异 (mutation) 从父代生成子代，以形成 $P(i+1)$ (Generate)
- $i \leftarrow i+1$
- 示例：
 - 我们用一个简单的进化算法 (种群大小 = 4) 来最大化函数 $f(x) = x^2$ ，其中 x 是整数，取值区间为 $[0,31]$
 - 种群大小 = 4 $\Leftrightarrow$ 有 4 个个体 / 染色体。
 - 编码与解码 (Encoding and decoding)
 - 表示：EA 应用的第一步是**编码** (即如何表示染色体)。
 - 这里采用二进制表示整数。
 - 使用 5 位二进制可以表示 0 ~ 31 的整数。
 - 示例：
 - 编码 31 $\rightarrow$ 二进制 11111 $\rightarrow$ 解码回 31 $\rightarrow$ 评价 $f(31) = 961$
 - 编码 16 $\rightarrow$ 二进制 10000 $\rightarrow$ 解码回 16 $\rightarrow$ 评价 $f(16) = 196$
 - 初始化：随机生成初始种群，例如 01101、11000、01000、10011



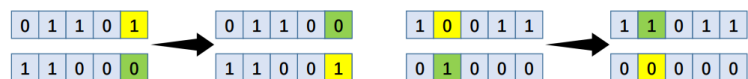
- 评估：为每个个体计算适应度。
 - a) 将个体解码为整数（称为表现型 phenotype）：
 - 01101 -> 13; 11000 -> 24; 01000 -> 8; 10011 -> 19
 - b) 按 $f(x)=x^2$ 计算适应度：
 - $f(13)=169$, $f(24)=576$, $f(8)=64$, $f(19)=361$
- 交叉 (Crossover)：
 - a) 根据个体的适应度选择两个个体进行交叉。
 - 如果使用轮盘赌选择 (roulette-wheel selection)，则第 i 个个体被选中的概率为：

$$P_i = \frac{f_i}{\sum_j f_j}$$

- f_i 是第 i 个个体的适应度，分母是所有个体适应度之和
- 通常每次交叉产生两个子代，并把它们加入中间种群；重复此步骤直到中间种群填满。
- 本例中的选择概率：

$$P_1(13) = \frac{169}{1170} = 0.14, P_2(24) = \frac{576}{1170} = 0.49, P_3(8) = \frac{64}{1170} = 0.06, P_4(19) = \frac{361}{1170} = 0.31$$

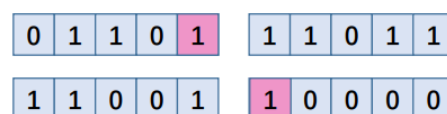
- b) 交叉示例
 - 单点交叉示意：
 - 将两个父本在某个位点切开，交换片段，得到两个子代。
 - 这就是在“选择 → 交叉”之后得到的中间候选子代集合。



Now the intermediate population is 01100, 11001, 11011, 00000.

- 对中间种群的个体以小概率应用变异 (mutation)。
 - 一种简单的变异是位翻转 (bit-flipping)。
 - 例如在本例中对某些位随机翻转后，得到新的种群 P(1)

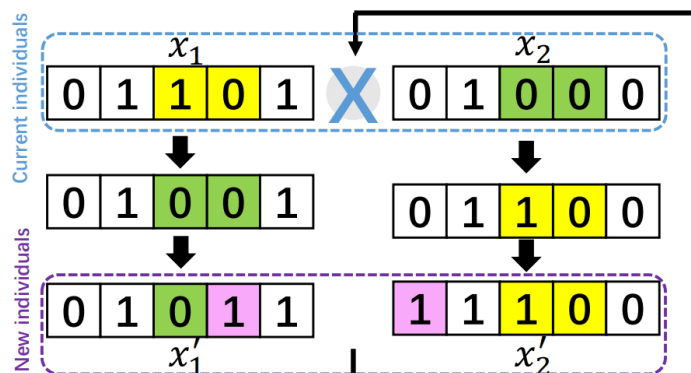
Example:



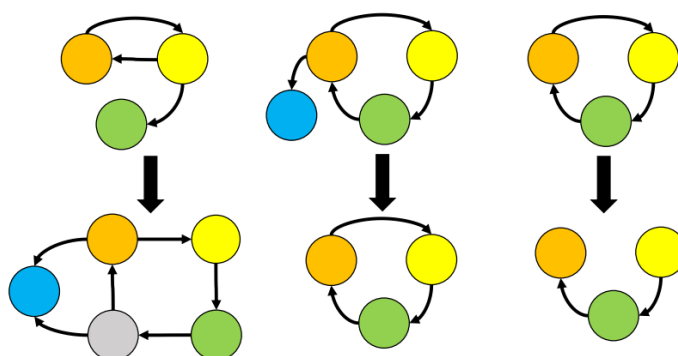
- 若未达到终止条件，返回步骤 2（评估）；否则结束并返回最优个体。
- 不同的进化算法

- 进化算法（EAs）指的是一整类算法家族，而不是单一算法。它们在历史背景、表示方法（representation）、变异/变体算子（variation operators）、选择方案（selection schemes）等方面各不相同。
- EA 家族：
 - 遗传算法（Genetic Algorithms, GAs）

Genetic Algorithms (GAs)



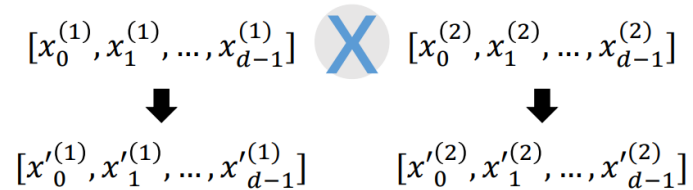
- 模拟达尔文式进化（Darwinian evolution）。
- 个体表示：二进制位串（Bit-strings）【染色体/chromosomes】
- 重组：单点交叉（1-Point crossover）
 - 强调重组（recombination / crossover）的作用；变异通常作为辅助背景算子使用。
- 变异：位翻转（Bit flip）
- 父代选择：按适应度比例（Fitness proportional）——用轮盘赌实现（Roulette Wheel）
- 生存者选择：按世代替换（Generational）
- 进化编程（Evolutionary Programming, EP）



- 更接近“拉马克式进化”（Lamarckian evolution）。
- 最初用有限状态机（FSM）表示个体；在数值优化中也常用实值向量表示。
- 搜索算子通常只是变异（mutations only），且作用于个体的表现型（phenotypic）表示。

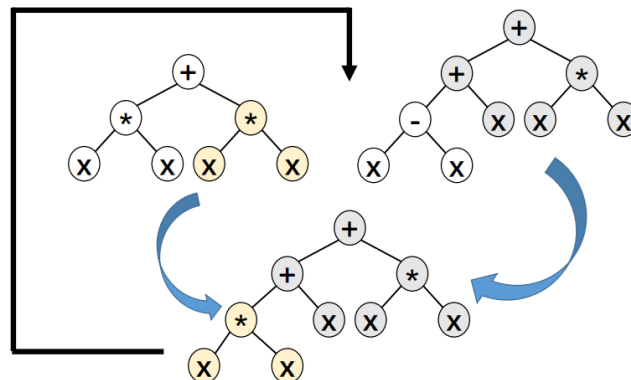
- 不使用重组
- 通常采用锦标赛选择 (tournament selection) 。
- 进化策略 (Evolution Strategies, ES)
 - 把每个个体看作一个 d 维实值向量

Evolution Strategies (ES)

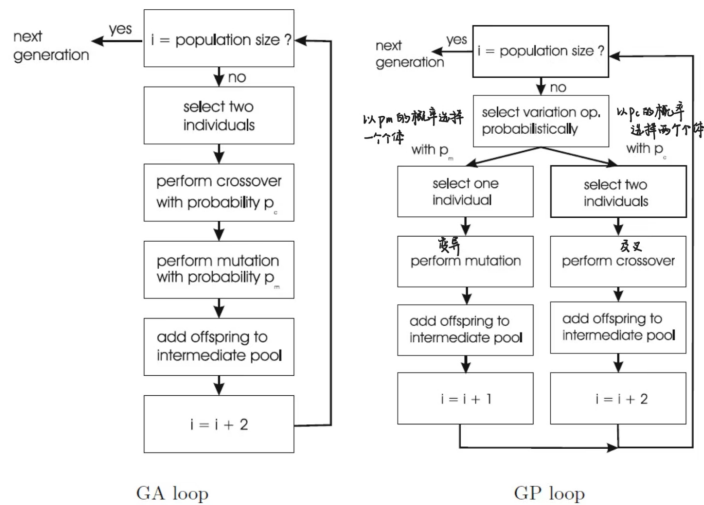


- 更接近拉马克式进化 (Lamarckian evolution) 。
- 个体表示：实值向量
- 重组：离散或中间式
- 变异：高斯扰动 【自适应变异 (self-adaptive mutations) 】
- 父代选择：均匀随机
- 生存者选择：以 (μ, λ) 或 $(\mu + \lambda)$ 的确定性精英替换
 - μ = 父代规模, λ = 子代规模
- 特性：变异步长的自适应
- 遗传编程 (Genetic Programming, GP)
 - 用树结构表示个体，交叉/重组通过交换子树生成新个体

Genetic Programming (GP)



- 同时使用交叉 (crossover) 和变异 (mutation) 。
-



- 更好且更通用的术语是使用“进化算法（EAs）”。

- 算子的变体（Variations in Operators）

- 交叉/重组（Crossover/Recombination）：单点交叉、两点交叉、均匀交叉、中间交叉等。
- 变异（Mutation）：位翻转（bit-flipping）、高斯变异（Gaussian mutation）、柯西变异（Cauchy mutation）等。
- 选择（Selection）：轮盘赌选择（按适应度比例选择）、锦标赛选择、基于排名的选择（线性与非线性）等。
- 替代策略（Replacement Strategy）：代际替换（generational）、稳态（连续）替换（steady-state / continuous）等。
- 特殊算子：多父重组（multi-parent recombination）、倒置（inversion）、基于序列的交叉（order-based crossover）等。

- 进化优化（Evolutionary Optimisation）

- 数值（全局）优化。
- 组合优化（用于 NP-困难问题）。
- 混合优化。
- 受约束的优化。
- 多目标优化。
- 在动态环境中的优化（具有动态适应度函数）。

- 进化学习（Evolutionary Learning）

- 进化学习可用于监督学习、无监督学习和强化学习。
- 学习分类器系统（基于规则的系统）。
- 进化人工神经网络。
- 进化模糊逻辑系统。
- 协同进化学习（Co-evolutionary learning）。

- 进化设计（Evolutionary Design）

- 进化计算技术特别擅长探索非常规设计，这些设计用手工很难获得。
- 人工神经网络的进化设计。
- 电子电路的进化设计。
- 可进化硬件（evolvable hardware）。
- （建筑）结构/建筑设计的进化设计。